```
import numpy as np
import matplotlib.pyplot as plt

# ==========================================
# 1. OPTIMIZATION ENVIRONMENT SETTINGS
# ==========================================
START = np.array([0, 0])
END = np.array([100, 100])
NUM_WAYPOINTS = 5  # Number of internal nodes in the drone path

# Obstacle definition: [center_x, center_y, radius]
OBSTACLE = np.array([50, 50, 20])

# ==========================================
# 2. FITNESS FUNCTION (THE SCORECARD)
# ==========================================
def calculate_fitness(path_waypoints):
    # Reconstruct full path: Start -> Waypoints -> End
    full_path = np.vstack([START, path_waypoints.reshape(NUM_WAYPOINTS, 2), END])

    # Objective 1: Minimize Total Path Length
    diffs = np.diff(full_path, axis=0)
    distance = np.sum(np.sqrt(np.sum(diffs**2, axis=1)))

    # Objective 2: Avoid Obstacles (Heavy Penalty if path crosses it)
    collision_penalty = 0
    # Check intermediate points along each segment for a simple collision check
    for i in range(len(full_path) - 1):
        p1 = full_path[i]
        p2 = full_path[i+1]
        # Check 10 points along each line segment
        for t in np.linspace(0, 1, 10):
            pt = p1 + t * (p2 - p1)
            dist_to_obs = np.linalg.norm(pt - OBSTACLE[:2])
            if dist_to_obs < OBSTACLE[2]:
                collision_penalty += 1000  # Large penalty to eliminate bad paths

    return distance + collision_penalty

# ==========================================
# 3. GREY WOLF OPTIMIZER (GWO)
# ==========================================
def grey_wolf_optimizer(num_wolves=30, max_iter=100):
    # Dimensions: Each wolf represents X and Y coordinates for our internal waypoints
    dim = NUM_WAYPOINTS * 2

    # Initialize wolf positions randomly between 0 and 100
    positions = np.random.uniform(0, 100, (num_wolves, dim))

    # Initialize Leader Scores
    alpha_score = float('inf')
    beta_score = float('inf')
    delta_score = float('inf')

    alpha_pos = np.zeros(dim)
    beta_pos = np.zeros(dim)
    delta_pos = np.zeros(dim)

    # Tracking history for visualization
    fitness_history = []

    # Main Optimization Loop
    for t in range(max_iter):
        for i in range(num_wolves):
            # Clip positions to keep them inside the 100x100 field boundaries
            positions[i] = np.clip(positions[i], 0, 100)

            # Calculate fitness
            fitness = calculate_fitness(positions[i])

            # Update Alpha, Beta, Delta
            if fitness < alpha_score:
                delta_score = beta_score; delta_pos = beta_pos.copy()
                beta_score = alpha_score; beta_pos = alpha_pos.copy()
                alpha_score = fitness; alpha_pos = positions[i].copy()
```

```
            elif fitness < beta_score:
                delta_score = beta_score; delta_pos = beta_pos.copy()
                beta_score = fitness; beta_pos = positions[i].copy()
            elif fitness < delta_score:
                delta_score = fitness; delta_pos = positions[i].copy()

        # Linearly decrease convergence factor 'a' from 2 to 0
        a = 2 - t * (2 / max_iter)

        # Update the position of all Omega wolves
        for i in range(num_wolves):
            for j in range(dim):
                # Pull towards Alpha
                r1, r2 = np.random.rand(), np.random.rand()
                A1 = 2 * a * r1 - a
                C1 = 2 * r2
                D_alpha = abs(C1 * alpha_pos[j] - positions[i, j])
                X1 = alpha_pos[j] - A1 * D_alpha

                # Pull towards Beta
                r1, r2 = np.random.rand(), np.random.rand()
                A2 = 2 * a * r1 - a
                C2 = 2 * r2
                D_beta = abs(C2 * beta_pos[j] - positions[i, j])
                X2 = beta_pos[j] - A2 * D_beta

                # Pull towards Delta
                r1, r2 = np.random.rand(), np.random.rand()
                A3 = 2 * a * r1 - a
                C3 = 2 * r2
                D_delta = abs(C3 * delta_pos[j] - positions[i, j])
                X3 = delta_pos[j] - A3 * D_delta

                # The safe zone compromise (average)
                positions[i, j] = (X1 + X2 + X3) / 3

        fitness_history.append(alpha_score)

    return alpha_pos.reshape(NUM_WAYPOINTS, 2), fitness_history

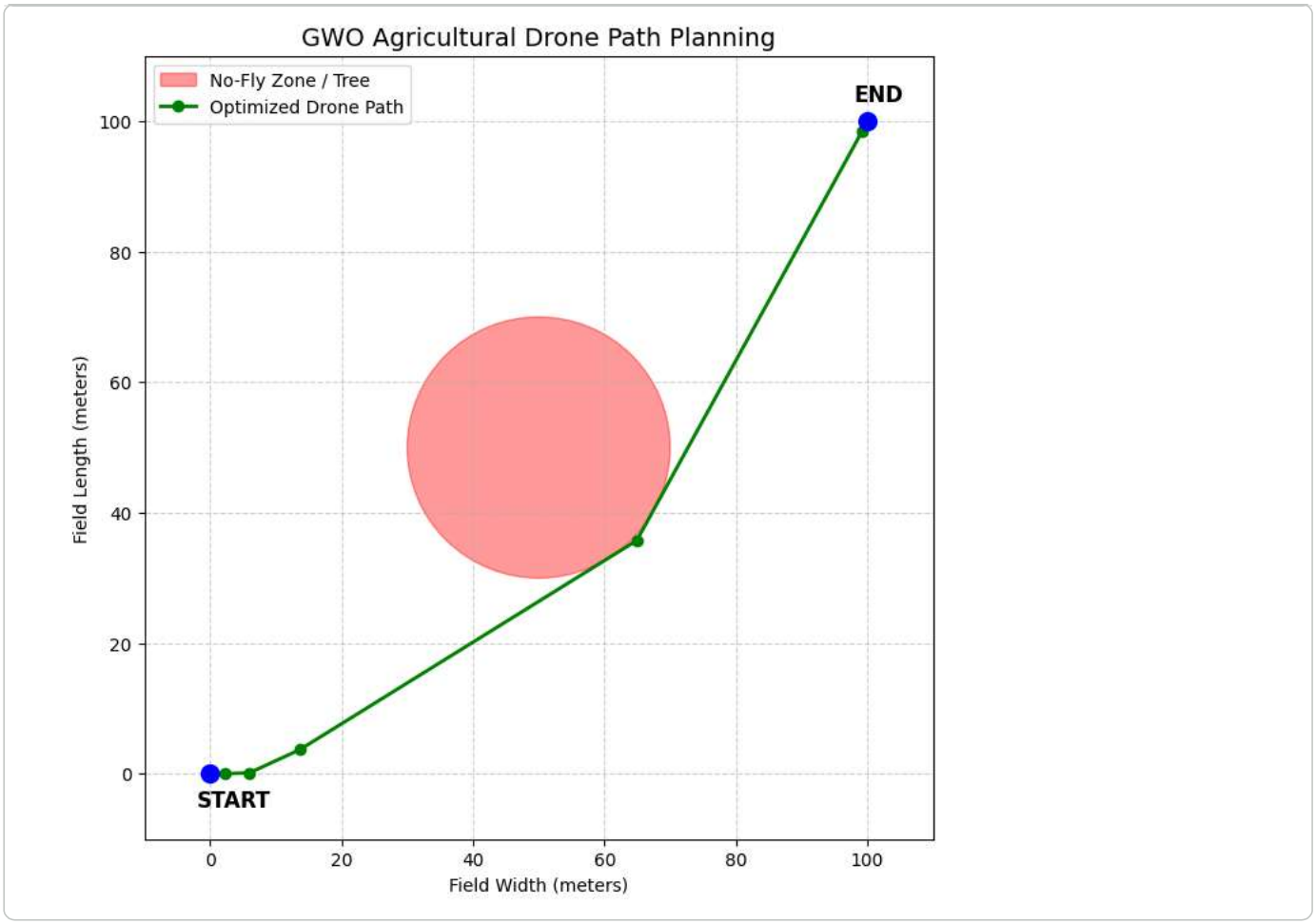
# ==========================================
# 4. EXECUTION & VISUALIZATION
# ==========================================
best_waypoints, history = grey_wolf_optimizer(num_wolves=40, max_iter=150)
final_path = np.vstack([START, best_waypoints, END])

# Plotting the map and final optimized route
plt.figure(figsize=(8, 8))
ax = plt.gca()

# Draw Obstacle (No-Fly Zone)
obstacle_circle = plt.Circle((OBSTACLE[0], OBSTACLE[1]), OBSTACLE[2], color='red', alpha=0.4, label='No-Fly Zone / Tree')
ax.add_patch(obstacle_circle)

# Plot start, end, and path
plt.plot(final_path[:, 0], final_path[:, 1], marker='o', color='green', linewidth=2, label='Optimized Drone Path')
plt.scatter([START[0], END[0]], [START[1], END[1]], color='blue', s=100, zorder=5)
plt.text(START[0]-2, START[1]-5, 'START', fontsize=12, fontweight='bold')
plt.text(END[0]-2, END[1]+3, 'END', fontsize=12, fontweight='bold')

plt.xlim(-10, 110)
plt.ylim(-10, 110)
plt.grid(True, linestyle='--', alpha=0.6)
plt.title('GWO Agricultural Drone Path Planning', fontsize=14)
plt.xlabel('Field Width (meters)')
plt.ylabel('Field Length (meters)')
plt.legend()
plt.show()
```


GWO Agricultural Drone Path Planning
No-Fly Zone / Tree
Optimized Drone Path
END
START
Field Length (meters)
Field Width (meters)
0
20
40
60
80
100